รายงานกลุ่มเรื่อง Tooket-ther

จัดทำขึ้นโดย

1. นางสาว นพัตธีรา เหลาเกิ้มหุ่ง 6710545010
2. นาย ปณิธาน ตันตื้อ 6710615144
3. นางสาว ลลิตา ทัศนอนันชัย 6710615243
4. นายพัชรพล มาลัยศรี 6710685055

เสนอ

รศ.ดร. วีรชัย อโณทัยไพบูลย์

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา วพ.230 ระบบฐานข้อมูล

ภาคเรียนที่ 2 ปีการศึกษา 2567

มหาวิทยาลัยธรรมศาสตร์ ศูนย์รังสิต

Overview

Tooket-ther คือแพลตฟอร์มบริหารจัดการการจองบัตรคอนเสิร์ตแบบครบวงจร (End-to-End Ticketing Platform) ที่ออกแบบมาเพื่อแก้ปัญหาความไม่เป็นธรรมในการเข้าถึงบัตรคอนเสิร์ตของคนในพื้นที่ และเพิ่มประสิทธิภาพในการจัดการข้อมูลหลังบ้าน (Back-end Management) สำหรับผู้จัดงาน ระบบถูกพัฒนาขึ้นโดยอาศัยหลักการจัดการระบบฐานข้อมูลเชิงสัมพันธ์ (Relational Database) เพื่อรองรับการทำธุรกรรม (Transactions) ที่มีความซับซ้อนและต้องการความแม่นยำสูง เช่น การจัดการคิว (Queue Management) การป้องกันการจองที่นั่งซ้ำซ้อน (Concurrency Control) และการประมวลผลการคืนเงิน (Refund Processing) นอกจากนี้ระบบยังให้ความสำคัญกับประสบการณ์ผู้ใช้งาน (UX) ทั้งในมุมของลูกค้าทั่วไป ผู้จัดงาน และพนักงานตรวจบัตร เพื่อให้กระบวนการตั้งแต่การซื้อจนถึงการเข้าชมงานเป็นไปอย่างราบรื่น

การวิเคราะห์ระบบและการออกแบบฐานข้อมูลในระยะเริ่มต้น (System Analysis and Initial Database Design)

ในการเริ่มต้นพัฒนาโครงการ **Tooket-ther** คณะผู้จัดทำได้ดำเนินการวิเคราะห์ความต้องการเชิงระบบและวางโครงสร้างพื้นฐานของฐานข้อมูล เพื่อให้รองรับกระบวนการทางธุรกิจ (Business Process) ที่ซับซ้อนได้อย่างมีประสิทธิภาพ โดยมีรายละเอียดดังนี้:

การวิเคราะห์กระบวนการทำงานหลัก (Core System Analysis)

จากการวิเคราะห์ Functional Requirement ระบบจำเป็นต้องจัดการกับข้อมูลที่มีความสัมพันธ์แบบพลวัต (Dynamic) สูง โดยแบ่งเป็น 3 ส่วนหลัก:

1. **การระบุสิทธิ์และลำดับ** (Identity & Priority Logic): ระบบต้องวิเคราะห์ข้อมูลภูมิลำเนาจากฐานข้อมูลผู้ใช้งาน เพื่อสร้างเงื่อนไขในการเข้าถึงข้อมูลตั๋ว (Access Control) ตามช่วงเวลาที่กำหนด
2. **การจัดการสถานะทรัพยากรที่นั่ง** (Inventory & Concurrency Control): วิเคราะห์สถานะที่นั่งแบบ Real-time โดยใช้หลักการจองแบบชั่วคราว (Soft Lock) เพื่อป้องกันปัญหาการเข้าถึงข้อมูลพร้อมกัน (Race Condition)
3. **การวิเคราะห์ข้อมูลเพื่อการตัดสินใจ** (Data Analytics for Organizer): ออกแบบกระบวนการดึงข้อมูล Transaction เพื่อสรุปผลรายรับและวิเคราะห์อัตราการจองในแต่ละโซน เพื่อใช้เป็นฐานข้อมูลในการตัดสินใจปิดโซนหรือปรับแผนการตลาด

การออกแบบโครงสร้างฐานข้อมูลเบื้องต้น (Initial Schema Design)

ในระยะเริ่มต้น คณะผู้จัดทำได้กำหนด **เอนทิตีหลัก** (Core Entities) และความสัมพันธ์เบื้องต้นเพื่อเป็นโครงสร้างในการจัดเก็บข้อมูล ดังนี้:

- **User Entity:** จัดเก็บข้อมูลพื้นฐานของผู้ใช้งาน (UserID, Name, Address, Email, PriorityStatus) โดยเน้นความสำคัญของฟิลด์ภูมิลำเนาเพื่อใช้จัดลำดับคิว

- **Concert & Zone Entity:** จัดเก็บข้อมูลงานคอนเสิร์ตและรายละเอียดโซนที่นั่ง (ZoneID, Price, TotalSeats, AvailableSeats) ซึ่งจะมีความสัมพันธ์แบบ One-to-Many กับโซนที่นั่ง
- **Seat Entity:** จัดเก็บข้อมูลเลขที่นั่งและสถานะปัจจุบัน (SeatID, ZoneID, Status: Available/Locked/Sold) เพื่อให้ระบบสามารถสืบค้นและอัปเดตสถานะได้รวดเร็ว
- **Booking & Transaction Entity:** จัดเก็บข้อมูลการทำรายการจอง (BookingID, UserID, SeatID, Timestamp, ExpiryTime) เพื่อใช้ในการควบคุมเวลาการจองและบันทึกประวัติการชำระเงิน
- **Payment Entity:** จัดเก็บหลักฐานและการยืนยันจากระบบภายนอก (PaymentID, BookingID, Amount, Method, Status)

feature requirement

User

1.ระบบต้องสามารถให้คนที่มีภูมิลำเนาในประเทศ มีสิทธิซื้อบัตรได้ก่อน
- ระบบจะดึงข้อมูลภูมิลำเนาในDatabaseมา โดยจะใช้เป็นkeyในการจำpriority

2.ระบบต้องสามารถคืนบัตรเมื่อมีเหตุฉุกเฉินได้ภายในเจ็ดวันก่อนวันคอนเสิร์ต
- User ทำการคืนบัตรเข้ามาในระบบและใส่ข้อมูลบัญชีธนาคารเพื่อใช้ในการรับเงินคืนโดยโดยจะสามารถรับเงินคืนได้แค่วิธีการโอนเข้าบัญชีธนาคารเพียงอย่างเดียว
- Adminจะทำการลบข้อมูลที่นั่งในDatabaseเพื่อให้ลูกค้าท่านอื่นสามารถจองที่นั่งได้

3.ระบบต้องสามารถ log in ผ่านบัญชี line และ facebook ได้
- ดึง API ภายนอกมาใช้ในการ log in

4.ระบบต้องสามารถดูสถานะที่นั่ง (ว่าง/ไม่ว่าง) ได้
- ข้อมูลในDatabase ที่แสดงให้ Userเห็นได้คือข้อมูลสถานะการจองที่นั่ง

5.ระบบต้องสามารถเลือกได้ว่าจะรับบัตรผ่านช่องทางไปรษณีย์ หรือรับที่หน้างานได้

6.ระบบต้องสามารถระบุชื่อผู้จองได้
- สามารถดึงkey name ออกมาใช้ได้

7.ระบบต้องสามารถจ่ายเงินผ่าน QR code ได้

8.ระบบต้องสามารถแสดงสถานะการจ่ายได้
- เมื่อuserชำระค่าบริการ ระบบจะบันทึกลงDatabaseและแสดงผลในหน้าตั๋วของฉัน โดยหากไม่ชำระเงินภายในเวลาที่กำหนด จะทำการยกเลิกการจองอัตโนมัติ

9.ระบบต้องสามารถแสดงการใช้งานของลูกค้าได้
- Userสามารถดูประวัติการซื้อของตนเองได้ในหน้าประวัติการซื้อตั๋ว โดยAdminจะดึงข้อมูลของUserมาแสดงผล

Organizer
1. ระบบต้องสามารถแสดงรายรับ และรายจ่ายจากการจัดคอนเสิร์ตได้
   - Database ดึงข้อมูลรายรับ/รายจ่าย ทั้งหมดมาแสดงผลเป็นตาราง
2. ระบบต้องสามารถจัดการระบบการเข้าคิวจ่ายเงินของผู้ซื้อได้

-Databaseจะดึงlocal keyมาจัดลำดับpriority และจะให้userเข้ามาจองตั๋วตามลำดับ โดยเมื่อที่นั่งถูกจองและชำระเงินเรียบร้อยแล้ว Databaseจะทำการปิดข้อมูลที่นั่งเพื่อไม่ให้ที่นั่งถูกจองซ้ำ

3. ระบบต้องสามารถพิจารณาปิดโซนเมื่อมีผู้ซื้อน้อยเกินไปได้
   -เมื่อโซนที่นั่งมีการจองที่น้อยเกินไปจะทำการปิดโซนที่นั่งDatabaseจะทำการลบข้อมูลในโซนนั่นๆ
4. ระบบต้องสามารถจัดการการย้ายโซน หรือการคืนเงินให้แก่ผู้ซื้อได้ เมื่อเกิดการปิดโซน
   -ปิดโซนที่นั่งโดยจะทำการแจ้งเตือนไปที่userและให้userเลือกว่าจะรับเงินคืนหรือจะอัพเกรดที่นั่งโดยเราจะให้userเข้าไปจองที่นั่งใหม่ได้ฟรี

Ticket checker
1.ระบบต้องสามารถให้คนตรวจบัตรเข้าถึงข้อมูลโซน และความถูกต้องของบัตรได้
-Checkerจะมีข้อมูลจากDatabaseเพื่อที่จะเช็คว่าข้อมูลของตั๋วตรงกับข้อมูลในDatabaseหรือไม่

Use case

1. User

1.1 use case: ลงทะเบียนภายในเว็บ
ผู้ใช้งานหลัก:
เงื่อนไขก่อนเริ่ม:
เหตุการณ์หลัก:
ผลลัพธ์:

1.2 use case: จองบัตรคอนเสิร์ต
ผู้ใช้งานหลัก: ลูกค้า
เงื่อนไขก่อนเริ่ม:
- ลูกค้าเข้าสู่ระบบเรียบร้อยแล้ว
- ลูกค้าเลือกโซนที่นั่ง ดูจำนวนและที่นั่งที่ว่าง

เหตุการณ์หลัก:
- ระบบแผนผังโซนที่นั่ง
- ลูกค้าดูที่นั่ง(ว่าง/ไม่ว่าง)
- เลือกที่นั่งที่ต้องการ
- ระบบตรวจสอบรายละเอียดที่นั่ง ความถูกต้องของข้อมูล
- ระบบชำระเงินพร้อมเลือกวิธีรับบัตร
- ระบบยืนยันการชำระเงิน
- ระบบตรวจสอบสถานะการชำระเงิน
- ระบบบันทึกข้อมูลการซื้อ

ผลลัพธ์:
- ระบบออกชื่อเจ้าของบัตร
- ระบบอัปเดตสถานะโซนและที่นั่ง
- ระบบบันทึกสถานะการชำระเงินเป็น “ชำระแล้ว”

2. Ticket checker
2.1 use case: ตรวจสอบและยืนยันบัตรคอนเสิร์ต

ผู้ใช้งานหลัก: ผู้ตรวจบัตรคอนเสิร์ต
เงื่อนไขก่อนเริ่ม: มีข้อมูลบัตรเรียบร้อยแล้ว
เหตุการณ์หลัก:
- ระบบตรวจสอบและยืนยันบัตรคอนเสิร์ตชื่อ โซนที่นั่ง ยูเซอร์ไอดี ถูกต้อง
- ตรวจสอบบัตรของลูกค้าผ่านการแสกนคิวอาร์โค้ด
- ขึ้นสถานะว่าบัตรถูกต้องหรือไม่

ผลลัพธ์: ระบบสามารถตรวจสอบและแสดงสถานะบัตรถูกต้องหรือไม่

3. Organizer

3.1 use case : ดูรายการและจำนวนรายรับรายจ่าย

ผู้ใช้งานหลัก: ผู้จัดงาน
เงื่อนไขก่อนเริ่ม:
- เข้าสู่ระบบเรียบร้อยแล้ว
- มีข้อมูลการจำนวนบัตรที่ถูกขาย รายได้และค่าใช้จ่าย

เหตุการณ์หลัก:
- ระบบดึงข้อมูลรายรับและรายจ่าย
- แสดงผลในรูปแบบสรุป (รายวัน/รวมทั้งหมด)

ผลลัพธ์:
- ผู้จัดงานสามารถดูและตรวจสอบรายรับรายจ่ายทั้งหมดของงาน

3.2 use case: ปิดโซนที่นั่งและย้ายผู้ที่ซื้อบัตรโซนนั้นไปแล้ว ไปโซนอื่น

ผู้ใช้งานหลัก: ผู้จัดงาน
เงื่อนไขก่อนเริ่ม: มีโซนที่มีจำนวนผู้ซื้อน้อยกว่าที่กำหนด
เหตุการณ์หลัก:
- ผู้จัดงานตรวจสอบจำนวนผู้ซื้อในแต่ละโซน
- ผู้จัดงานยืนยันเลือกโซนที่ต้องปิดเนื่องจากจำนวนไม่พอ
- ระบบย้ายผู้ซื้อบัตรในโซนที่ปิดไปยังอีกโซนที่ว่าง
- ระบบยืนยันการปิดโซนที่นั่ง
- ระบบดำเนินการปิดโซนที่นั่ง
- ระบบบอกรายละเอียดและแจ้งเตือนผ่านทางอีเมล
-

ผลลัพธ์:
- โซนที่นั่งถูกปิด
- ผู้ซื้อได้รับทราบข้อมูลและตรวจสอบความถูกต้องของบัตรคอนเสิร์ต

3.3 use case: จัดการคิวชำระเงิน

ผู้ใช้งานหลัก: ผู้จัดงาน
เงื่อนไขก่อนเริ่ม: มีลูกค้าที่กำลังดำเนินการซื้อบัตรและรอชำระเงิน
เหตุการณ์หลัก:
- ระบบสร้างคิวการชำระเงินอัตโนมัติ
- ระบบแสดงลำดับผู้ชำระเงิน
- ระบบจัดการสิทธิ์การชำระเงินตามคิว

ผลลัพธ์:
- ระบบจัดการคิวชำระเงินจัดลำดับถูกต้องเป็นระเบียบและไม่มีข้อมูลตกหล่น

Stakeholder

ลูกค้า : ดูราคาของที่นั่งแต่ละโซน เลือกโซนที่นั่งคอนเสิร์ต สามารถดูสถานะว่างหรือไม่ว่างของแต่ละโซนและจำนวนเก้าอี้ที่เหลือ ดูรายละเอียดก่อนซื้อบัตรคอนเสิร์ต สามารถเลือกวิธีชำระเงินได้ เลือกวิธีการรับบัตร ยืนยันการซื้อบัตร ดูประวัติและรายละเอียดการซื้อ คืนบัตรภายในเจ็ดวันหากไม่สามารถไปได้แล้ว

ผู้จัดงาน: ดูจำนวนที่ว่างและบัตรของแต่ล่ะโซนได้ ตรวจสอบรายรับรายจ่ายภายในงานได้ หากมีการปิดโซนสามารถย้ายคนในโซนนั้นไปโซนอื่นได้

พนักงานตรวจบัตร: ตรวจสอบและยืนยันความถูกต้องบัตรของคนเข้างาน

สปอนเซอร์: ผู้สนับสนุนและตรวจสอบเอนเกจของคอนเสิร์ต

Pain point

1. **ความเหลื่อมล้ำในการเข้าถึงบัตร (Local Access Issue):** แฟนคลับที่มีภูมิลำเนาในประเทศที่จัดงานมักประสบปัญหากดจองบัตรไม่ทัน เนื่องจากมีชาวต่างชาติหรือกลุ่มคนรับจ้างกดบัตรจากต่างประเทศเข้ามาแย่งโควตา
2. **ปัญหาเว็บล่มและการจองซ้ำซ้อน (Concurrency & System Overload):** เมื่อมีผู้ใช้งานจำนวนมากเข้ามาทำรายการพร้อมกัน (High Traffic) ระบบมักจะล่ม หรือเกิดข้อผิดพลาดทางฐานข้อมูลที่ทำให้ที่นั่งเดียวกันถูกขายให้ลูกค้ามากกว่า 1 คน (Double Booking)
3. **ปัญหาการกักตุนบัตรเพื่อโก่งราคา (Scalpers & Bots):** ขาดระบบการยืนยันตัวตนและการผูกชื่อเจ้าของบัตรที่เข้มงวด ทำให้มีการใช้บอทกวาดซื้อบัตรไปขายต่อในราคาที่สูงกว่าปกติ (บัตรผี)
4. **ความไม่ยืดหยุ่นเมื่อเกิดเหตุฉุกเฉิน (Inflexibility in Ticket Management):** ผู้ซื้อบัตรไม่มีช่องทางในการคืนบัตรอย่างเป็นระบบหากติดธุระฉุกเฉิน และในฝั่งผู้จัดงานก็จัดการย้ายโซนหรือคืนเงินได้ยากเมื่อมีปัญหาบัตรขายได้น้อยเกินไปในบางโซน

Goals (เป้าหมายของโครงการ)

เพื่อพัฒนาระบบการจัดสรรสิทธิ์ที่เที่ยงธรรม: ออกแบบระบบฐานข้อมูลที่สามารถคัดกรองและให้สิทธิพิเศษ (Priority Queue) แก่ผู้ใช้งานที่มีภูมิลำเนาในพื้นที่จัดการแสดงเป็นลำดับแรก

เพื่อสร้างระบบฐานข้อมูลที่มีเสถียรภาพและแม่นยำสูงสุด: พัฒนาโครงสร้างฐานข้อมูลที่รองรับคุณสมบัติ ACID (Atomicity, Consistency, Isolation, Durability) อย่างสมบูรณ์ เพื่อป้องกันปัญหาระบบขัดข้องและขจัดข้อผิดพลาดจากการจองที่นั่งซ้ำซ้อน ภายใต้สภาวะที่มีผู้ใช้งานพร้อมกันจำนวนมาก

เพื่อเพิ่มความยืดหยุ่นในการบริหารจัดการวงจรชีวิตของบัตร (Ticket Lifecycle Management): พัฒนาระบบที่รองรับการคืนสิทธิ์ (Refund) ตามเงื่อนไขที่กำหนด และสนับสนุนผู้จัดงานในการปรับปรุงข้อมูลสถานะที่นั่ง การปิดโซน หรือการย้ายโซนที่นั่งได้แบบทันที (Real-time)

เพื่อป้องกันการจำหน่ายบัตรเกินราคาและการกระทำผิดกฎหมาย: บังคับใช้ระบบการยืนยันตัวตนและระบุชื่อผู้จองลงในฐานข้อมูล พร้อมทั้งบูรณาการระบบตรวจสอบความถูกต้องผ่านรหัสคิวอาร์ (QR Code) ณ จุดคัดกรองเพื่อยืนยันสิทธิ์ก่อนเข้างาน

ข้อสมมติฐานของระบบ (Assumptions)

เพื่อให้การออกแบบระบบใน Iteration แรกมีความชัดเจน คณะผู้จัดทำจึงได้กำหนดข้อสมมติฐานเบื้องต้นไว้ดังนี้:

- **Queue Prioritization Logic :** สมมติให้ระบบฐานข้อมูลสามารถประมวลผลและจัดลำดับคิว (Sequential Processing) ได้อย่างแม่นยำ โดยใช้ข้อมูลภูมิลำเนา (Address/Location) ที่ดึงมาจากฐานข้อมูลทะเบียนราษฎร์หรือข้อมูลการลงทะเบียนของผู้ใช้งาน เป็นตัวแปรหลัก (Key Attribute) ในการกำหนดลำดับสิทธิ์ (Priority Score) เพื่อให้ผู้มีภูมิลำเนาในประเทศเข้าสู่ขั้นตอนการเลือกที่นั่งได้ก่อนกลุ่มอื่น
- **Queue Persistence:** สมมติว่าเมื่อผู้ใช้งานเข้าสู่ระบบคิวแล้ว ข้อมูลลำดับคิวจะถูกบันทึกและคงสถานะไว้ในหน่วยความจำชั่วคราวหรือฐานข้อมูล (Session/Cache Management) เพื่อป้องกันการสูญเสียลำดับคิวในกรณีที่การเชื่อมต่ออินเทอร์เน็ตของผู้ใช้งานเกิดการขัดข้องชั่วคราว
- **Payment Gateway Integration:** ระบบสมมติให้มีการเชื่อมต่อกับ API ภายนอกสำหรับการชำระเงินผ่าน QR Code และได้รับการยืนยันสถานะการชำระเงินกลับมายังฐานข้อมูลแบบ Real-time
- **Single Source of Truth:** ฐานข้อมูลกลางจะเป็นแหล่งข้อมูลเดียวที่ใช้ยืนยันสถานะที่นั่ง (ว่าง/ไม่ว่าง) เพื่อป้องกันความขัดแย้งของข้อมูลระหว่างฝั่ง User และ Organizer
- **User Identification:** ผู้ใช้งานหนึ่งรายสามารถผูกบัญชีได้เพียงหนึ่งประเภท (Line หรือ Facebook) เพื่อป้องกันความซ้ำซ้อนในการจองคิว

ปัญหาที่คาดว่าจะพบ (Anticipated Challenges)

1. **ความซับซ้อนในการประมวลผลลำดับคิวตามเงื่อนไข (Complex Priority Queue Processing):** การจัดลำดับคิวโดยมีเงื่อนไขด้านภูมิลำเนา (Location-based Priority) อาจส่งผลให้การ Query ข้อมูลมีความซับซ้อนสูง (High Query Complexity) ซึ่งหากระบบฐานข้อมูลไม่ได้มีการทำดัชนี (Indexing) ที่มีประสิทธิภาพ อาจทำให้เกิดความล่าช้าในการจัดลำดับผู้ใช้งานจำนวนมากพร้อมกัน
2. **การจัดการคิวหลุดหรือการยกเลิกธุรกรรม (Queue Expiry & Session Timeout):** ปัญหาในการตัดสินใจของระบบฐานข้อมูลเมื่อผู้ใช้งานในลำดับคิวต้นๆ ไม่ดำเนินการจองภายในเวลาที่กำหนด ระบบต้องมีการจัดการ Rollback Transaction และเลื่อนลำดับคิวถัดไปขึ้นมาโดยอัตโนมัติอย่างแม่นยำ เพื่อไม่ให้เสียโอกาสทางธุรกิจและลดความพึงพอใจของผู้ใช้
3. **Race Condition:** ปัญหาการแย่งชิงทรัพยากรข้อมูลในเวลาเดียวกัน (Concurrent Access) เช่น ผู้ใช้งานหลายคนพยายามกดจองที่นั่งตัวเดียวกันในเวลาเสี้ยววินาที ซึ่งอาจส่งผลให้สถานะที่นั่งในฐานข้อมูลผิดพลาดหากไม่มีการจัดการ Transaction ที่ดีพอ
4. **API Latency & Failure:** ความล่าช้าหรือความล้มเหลวในการดึงข้อมูลจาก Third-party API (Line, Facebook, Payment Gateway) ซึ่งอาจทำให้การ Log-in หรือการปรับปรุงสถานะการจ่ายเงินใน Database เกิดความล่าช้า
5. **Data Security & Privacy:** การจัดเก็บข้อมูลส่วนบุคคลและข้อมูลบัญชีธนาคารสำหรับการคืนเงิน จำเป็นต้องมีมาตรการรักษาความปลอดภัยที่เข้มงวดเพื่อป้องกันการรั่วไหลของข้อมูล
6. **Database Scaling:** ในกรณีที่มีการเปิดขายบัตรคอนเสิร์ตขนาดใหญ่ ฐานข้อมูลต้องรองรับ Query ปริมาณมหาศาลในระยะเวลาอันสั้น (Peak Load) ซึ่งอาจส่งผลต่อ Performance ของระบบ

7. แผนการดำเนินงานต่อไป (Future Work Plan)

ระยะที่ 1: การพัฒนาส่วนเชื่อมต่อและระบบจัดการคิว (System Integration & Queue Implementation)

- ดำเนินการสร้างหน้าจอส่วนต่อประสาน (UI) สำหรับการลงทะเบียนและจองบัตร
- พัฒนาระบบตรรกะเบื้องหลัง (Back-end Logic) ในการจัดลำดับคิวโดยอ้างอิงจากข้อมูลภูมิลำเนาในฐานข้อมูล เพื่อคัดกรองผู้ใช้งานตามเงื่อนไขที่กำหนด

ระยะที่ 2: การพัฒนาระบบชำระเงินและจัดการสถานะที่นั่ง (Payment & Seat Management)

- เชื่อมต่อระบบการชำระเงินผ่าน QR Code และเขียนคำสั่งเพื่ออัปเดตสถานะที่นั่งในฐานข้อมูลโดยอัตโนมัติเมื่อมีการทำธุรกรรมสำเร็จ
- พัฒนาระบบตรวจสอบเวลา (Timer) เพื่อคืนสถานะที่นั่งให้เป็น "ว่าง" หากผู้ใช้งานไม่ดำเนินการชำระเงินภายในเวลาที่กำหนด
- ระบบบริหารจัดการปิดโซนที่นั่ง (Zone Closure & Seat Consolidation)พัฒนาระบบสำหรับผู้จัดงาน (Organizer) ให้สามารถตรวจสอบความหนาแน่นของผู้จองในแต่ละโซน และมีคำสั่งสำหรับการ "ปิดโซน" (Zone Deactivation) ในกรณีที่มีจำนวนผู้ซื้อน้อยกว่าเกณฑ์ที่กำหนด

ระยะที่ 3: การพัฒนาระบบสำหรับผู้จัดงานและเจ้าหน้าที่ (Organizer & Checker Tools)

- สร้างระบบสรุปรายงานรายรับ-รายจ่าย (Dashboard) สำหรับผู้จัดงานโดยการดึงข้อมูล Transaction จากฐานข้อมูล
- พัฒนาส่วนการตรวจสอบบัตร (Ticket Verification) สำหรับเจ้าหน้าที่หน้างาน เพื่อยืนยันความถูกต้องของบัตรผ่านการสแกน QR Code

ระยะที่ 4: การทดสอบระบบและความถูกต้องของข้อมูล (System Testing & Validation)

- ทดสอบการทำงานของระบบในสภาวะที่มีการจองพร้อมกัน (Concurrency Testing) เพื่อตรวจสอบความแม่นยำของสถานะที่นั่งและการลำดับคิว
- ตรวจสอบความถูกต้องของการบันทึกประวัติการซื้อ (Purchase History) และการจัดการข้อมูลกรณีมีการคืนบัตรหรือปิดโซนที่นั่ง